SDELP-DDPG 논문 구현 해석

Stochastic Differential Equations with Lévy Processes-Driven
Deep Deterministic Policy Gradient for Portfolio Management

Session 3: Python/PyTorch 코드 구현 상세 해석

사전 지식: Session 1, Session 2

강의 시간: 약 60분

Contents

1 개요 (Overview) 3

1.1 Session 3의 목표 . . . 3

1.2 코드베이스 구조 . . . 3

1.3 데이터 흐름과 논문 매핑 . . . 3

2 설정 모듈: `config.yaml` + `config.py` 4

2.1 `config.yaml` — 사용자 설정 . . . 4

2.2 `config.py` — 하이퍼파라미터 . . . 4

3 포트폴리오 환경: `environment.py` (식 (1)∼(6)) 5

3.1 데이터 로드 및 상대 가격 벡터: 식 (1) . . . 5

3.2 상태 공간 (State) 구성 . . . 5

3.3 포트폴리오 가중치 진화: 식 (2) . . . 6

3.4 거래 비용: 식 (3) . . . 6

3.5 즉시 보상 (Immediate Reward): 식 (4)∼(5) . . . 7

3.6 상대 엔트로피 기반 보상 함수: 식 (6) . . . 7

4 네트워크 구조: `networks.py` (식 (7)∼(9), Fig. 2, Fig. 3) 8

4.1 Lévy 노이즈 생성기: Definition 2.1 . . . 8

4.2 Conv2D 특성 추출기: Fig. 2의 초기 행동 생성 . . . 9

4.3 Drift Net과 Diffusion Net: 식 (7)의 핵심 구성 요소 . . . 10

4.4 SDE Actor: 식 (8)의 전체 구현 . . . 11

4.5 Critic 네트워크: 논문 Fig. 3 . . . 13

5 DDPG 에이전트: `agent.py` (식 (10)∼(17), Algorithm 1) 14

5.1 Ornstein-Uhlenbeck 탐색 노이즈: 식 (16) 14
5.2 경험 리플레이 버퍼: Algorithm 1 Line 14 15
5.3 에이전트 초기화: Algorithm 1 Line 1∼3 16
5.4 행동 선택: Algorithm 1 Line 7∼12 (식 (8) + 식 (16)) 16
5.5 학습 업데이트: 식 (15) + 식 (17) — Algorithm 1의 핵심 17
5.6 소프트 타겟 업데이트: Algorithm 1 Line 20 18

6 학습 루프: `train.py` (Algorithm 1 외부 루프) 20
6.1 학습 메인 루프: Algorithm 1 Line 4∼21 20
6.2 모델 저장 21

7 백테스트: `backtest.py` (식 (19), Table 2, Table 3) 21
7.1 평가지표 계산: 논문 식 (19) 22
7.2 백테스팅 실행 흐름 24

8 Lévy vs Gaussian 비교: `compare_levy_gaussian.py` 25
8.1 노이즈 분포 비교: 논문 Table 1의 시각화 25
8.2 행동 분포 비교 25

9 통합 실행: `run_all.py` 26

10 출력 결과 해석 26
10.1 Figure 1: `levy_vs_gaussian.png` — Lévy vs Gaussian 노이즈 비교 26
10.2 Figure 2: `action_distribution.png` — 행동 분포 비교 28
10.3 Figure 3: `training_results.png` — 학습 결과 4-Panel 28
10.4 Figure 4: `losses.png` — Actor/Critic Loss 30
10.5 Figure 5: `backtest_results.png` — 백테스트 결과 (별도 실행) 31

11 핵심 기여점 및 토론 32
11.1 수식에서 코드로의 핵심 변환 요약 32
11.2 구현상의 핵심 설계 판단 32
11.3 Session 2의 한계점과 코드에서의 대응 32

1 개요 (Overview)

1.1 Session 3의 목표

Session 2에서 분석한 논문의 수식 체계(식 (1)∼(18), Algorithm 1)가 Python/PyTorch로 어떻게 구현되는지를 코드 레벨에서 상세히 해석합니다.

Session 2 → Session 3 연결

- Session 2: 논문의 **수학적 정식화** — 수식 (1)∼(18)의 유도와 해석
- Session 3: 각 수식이 **코드의 어느 부분**에, **어떻게** 구현되었는지 분석
- 목표: "수식을 보고 코드를 읽을 수 있고, 코드를 보고 수식을 떠올릴 수 있는" 상태 달성

1.2 코드베이스 구조

프로젝트 파일 구조

```
sdelp-ddpg/
├── config.yaml              # 실험 설정 (기간, 에피소드, 종목)
├── config.py                # 하이퍼파라미터 로드 + 상수 정의
├── environment.py           # 포트폴리오 MDP 환경 (식 1~6)
├── networks.py              # SDE Actor + Critic 네트워크 (식 7~8, Fig.2~3)
├── agent.py                 # DDPG 에이전트 (식 10~17, Algorithm 1)
├── train.py                 # 학습 루프 + 시각화
├── backtest.py              # 테스트 기간 평가 (식 19)
├── compare_levy_gaussian.py # Lévy vs Gaussian 비교 시각화
├── run_all.py               # 전체 파이프라인 실행
└── outputs/                 # 결과 저장
```

1.3 데이터 흐름과 논문 매핑

Table 1: 코드 파일과 논문 수식/섹션 매핑

코드 파일	논문 수식	논문 섹션
`config.py`	하이퍼파라미터 전체	Section 4.1
`environment.py`	식 (1)∼(6)	Section 3.1
`networks.py`	식 (7)∼(9), Fig. 2, Fig. 3	Section 3.2
`agent.py`	식 (10)∼(17), Algorithm 1	Section 3.3
`train.py`	Algorithm 1 외부 루프	Section 4
`backtest.py`	식 (19), Table 2, Table 3	Section 4.2∼4.3

2 설정 모듈: config.yaml + config.py

2.1 config.yaml — 사용자 설정

사용자가 직접 수정하는 유일한 파일입니다. 학습/테스트 기간, 에피소드 수, 투자 종목을 정의합니다.

Listing 1: config.yaml

```yaml
# 학습테스트/ 기간
train_start: "2018-01-01"
train_end: "2022-12-31"
test_start: "2023-01-01"
test_end: "2026-01-31"
# Trajectory에피소드() 수
num_episodes: 10
# 종목리스트 (Yahoo Finance 티커)
tickers:
  - "BTC-USD"
  - "DOGE-USD"
  - "ETH-USD"
  # ... 총( 개9 암호화폐)
```

2.2 config.py — 하이퍼파라미터

config.yaml을 로드하고, 논문의 핵심 하이퍼파라미터를 상수로 정의합니다. 모든 모듈이 from config import *로 이 값들을 사용합니다.

Listing 2: config.py — 핵심 하이퍼파라미터

```python
# 포트폴리오환경
WINDOW_SIZE = 50        # 관측윈도우일 ()
TRANSACTION_COST = 0.0025 # 거래비용비율논문(: 0.25%)
BETA = 0.05             # 리스크페널티가중치
GAMMA = 0.99            # 할인율
# SDE Actor 네트워크
SDE_STEPS = 5           # Euler-Maruyama 이산화스텝수논문(: m)
LEVY_ALPHA = 1.4        # alpha-stable 분포파라미터
LEVY_BETA_PARAM = 0.0 # 대칭분포 (beta = 0)
DRIFT_HIDDEN = 64       # Drift Net 은닉층크기
DIFFUSION_HIDDEN = 64 # Diffusion Net 은닉층크기
# DDPG 학습
BUFFER_SIZE = 50000     # 리플레이버퍼용량
BATCH_SIZE = 64         # 미니배치크기
ACTOR_LR = 1e-4         # Actor 학습률
CRITIC_LR = 3e-4        # Critic 학습률
TAU = 0.001             # 소프트업데이트계수
```

논문 Table 1과의 대응

- LEVY_ALPHA = 1.4: 암호화폐 시장에 =1.4가 최적 (논문 Table 1). 변동이 클수록 작은 사용
- BETA = 0.05: 논문 Fig. 7에서 =0.05일 때 최대 보상 달성. Markowitz(1952)와 일치
- TAU = 0.001: 매 업데이트마다 Target 네트워크의 0.1%만 교체 (Session 2 식 (17) 참조)

3 포트폴리오 환경: environment.py (식 (1)∼(6))

PortfolioEnv 클래스는 논문 Section 3.1의 MDP 정의를 구현합니다. 이 클래스는 논문 전체에서 에이전트가 상호작용하는 **시장 시뮬레이터** 역할을 합니다.

3.1 데이터 로드 및 상대 가격 벡터: 식 (1)

논문 식 (1) — 상대 가격 벡터

$$v^t := \frac{\mathbf{x}^t}{\mathbf{x}^{t-1}} = \left(1, \frac{x_1^t}{x_1^{t-1}}, \ldots, \frac{x_n^t}{x_n^{t-1}}\right)^\top$$

첫 번째 원소 1은 현금 자산 (가격 변동 없음). 각 원소는 전일 대비 가격 변화율.

Listing 3: environment.py — 상대 가격 벡터 계산 (식 (1))

```python
def _load_data(self, tickers, start, end):
    """Yahoo 에서Finance  가격데이터로드"""
    df = yf.download(tickers, start=start, end=end, auto_adjust=True)
    close = df["Close"]
    close = close.ffill().bfill() # 결측치처리

    prices = close.values # (T, num_assets)

    # 상대가격변화율  : v_t = p_t / p_{t-1} <-- 식(1)
    returns = prices[1:] / prices[:-1] # (T-1, num_assets)

    return prices, returns, dates
```

구현 세부사항

- 식 (1)의 첫 번째 원소 1 (현금)은 데이터 로드 단계에서 포함하지 않고, `step()` 메서드에서 `np.concatenate([[1.0], v_t])`로 동적 추가
- `ffill().bfill()`: 논문의 "인접 값 평균으로 대체" 전략의 간소화 구현
- 데이터 부족 종목은 자동 제외 — 이 경우 `state_dim`과 `action_dim`이 동적으로 변경됨

3.2 상태 공간 (State) 구성

Listing 4: environment.py — State 구성

```python
def _get_state(self):
    """State: 가격변화율윈도우+  현재포트폴리오가중치"""
    start = self.step_idx
    end = start + self.window
    price_window = self.returns[start:end] # (50, 9)

    state = np.concatenate([
        price_window.flatten(), # 50 * 9 = 450 차원
        self.weights          # 10 차원현금( + 9 자산)
    ])
    return state.astype(np.float32) # 총460 차원
```

State 차원 분해

$$\text{state_dim} = \underbrace{n \times w}_{450} + \underbrace{(n+1)}_{10} = 460$$

여기서 $n = 9$ (자산 수), $w = 50$ (관측 윈도우). 논문에서는 상태 v^t를 고정 윈도우 내의 가격 정보 텐서로 정의하며, 현재 가중치 w^{t-1}를 추가로 포함합니다.

3.3 포트폴리오 가중치 진화: 식 (2)

논문 식 (2) — 가중치 진화

$$w^{t-1} = \frac{a^{t-1} \odot (v^{t-1})^\top}{a^{t-1} \cdot v^{t-1}}$$

시장 가격 변동에 의해 포트폴리오 가중치가 자연스럽게 변화합니다.

Listing 5: environment.py — 가중치 진화 (식 (2))

```
# step() 메서드내부

# v_full = (1.0, v_t[0], v_t[1], ...) --- 현금포함수익률벡터
v_full = np.concatenate([[1.0], v_t])

# 가중치업데이트거래 ( 후시장변동반영 ) --- 식(2)
# w_new * v_full = a^{t-1} ⊙v^{t-1} 요소별( 곱)
# np.dot(v_full, w_new) = a^{t-1} · v^{t-1} 내적( = 총수익률)
self.weights = (w_new * v_full) / max(np.dot(v_full, w_new), 1e-8)
```

식 (2)는 "행동으로 설정한 가중치가 시장 변동에 의해 자연스럽게 드리프트한다"는 현실적 가정을 구현합니다. 가격이 더 많이 오른 자산의 비중이 자연스럽게 증가합니다.

3.4 거래 비용: 식 (3)

논문 식 (3) — 상대적 거래 비용

$$u_c(t) = \varphi \times \frac{\|w^t - w^{t-1}\|}{w^t \cdot v^t} \times v^t$$

본 구현에서는 간소화하여 $\mu_t = 1 - c_s \times \sum |w_{\text{new}} - w_{\text{old}}|$ 형태로 적용합니다.

Listing 6: environment.py — 거래 비용 (식 (3)의 간소화)

```
def _transaction_cost(self, w_old, w_new):
    """거래비용모델논문
      ( Eq. 3)
    mu_t = 1 - c_s * sum|w_new - w_old|
    """
    turnover = np.sum(np.abs(w_new - w_old))
    mu = 1.0 - self.tc_rate * turnover # tc_rate = 0.0025
    return max(mu, 0.0)
```

논문과 구현의 차이

논문 식 (3)은 상대적 거래 비용 $u_c(t) = \varphi \frac{\|w^t - w^{t-1}\|}{w^t \cdot v^t} \times v^t$ 으로, 시장 변동이 클 때도 안정적인 비용을 산출합니다. 구현에서는 이를 $\mu_t = 1 - \varphi \sum |\Delta w|$ 로 간소화하여, 턴오버(가중치 변화량)에 비례하는 비용을 포트폴리오 수익률에 곱하는 방식을 사용합니다. 경제적 의미는 동일합니다: 재배분이 클수록 비용이 증가합니다.

3.5 즉시 보상 (Immediate Reward): 식 (4)∼(5)

논문 식 (4)∼(5) — 로그 수익률

$$r_t = \log\left(a^{t-1} \cdot v^{t-1} - u_c(t)\right)$$

로그 수익률을 사용하는 이유: (1) 오목성으로 위험 회피 성향 반영, (2) 다기간 수익률의 덧셈 용이성.

Listing 7: environment.py — 즉시 보상 (식 (4)∼(5))

```
# 포트폴리오수익률논문 ( Eq. 와2 거래비용결합)
portfolio_return = mu_t * np.dot(v_full, w_new)

# 로그수익률논문 ( Eq. 4~5)
log_return = np.log(max(portfolio_return, 1e-8))
```

3.6 상대 엔트로피 기반 보상 함수: 식 (6)

논문 식 (6) — 최종 목적 함수 (Session 2의 핵심)

$$R = \sum_t \gamma^t \Big(r_t(v^t, a^t) - \beta\, D(w^t \| w^{t-1}) \Big)$$

$\beta > 0$은 위험 페널티의 가중치 파라미터. D는 KL divergence.

Listing 8: environment.py — 상대 엔트로피 + 최종 보상 (식 (6))

```
def _relative_entropy(self, w_new, w_old):
    """상대엔트로피 D_KL(w_new||w_old) = sum(w_new * ln(w_new/w_old))"""
    w_new_c = np.clip(w_new, 1e-8, 1.0)
    w_old_c = np.clip(w_old, 1e-8, 1.0)
    return np.sum(w_new_c * np.log(w_new_c / w_old_c))

# step() 메서드내부 --- 최종보상식 (6)
kl_penalty = self._relative_entropy(w_new, w_old)
reward = log_return - self.beta * kl_penalty
# R_t = r_t - beta * D_KL(w_new || w_old)
```

식 (6)의 역할 — 논문 전체에서의 위치

식 (6)은 논문의 **보상 설계**의 핵심입니다:

- **수익 항** (r_t): 에이전트가 높은 수익을 추구하도록 유도
- **KL 페널티** ($\beta \cdot D$): 포트폴리오 급변을 억제하여 안정적 투자 전략 유도
- 이 보상은 식 (18)의 통합 최적화 목적함수이며, Actor-Critic 학습(식 (10)∼(17))의 기반

4 네트워크 구조: `networks.py` (식 (7)∼(9), Fig. 2, Fig. 3)

`networks.py`는 논문의 핵심인 **SDE 기반 Actor**와 **다층 합성곱 잔차 Critic**을 구현합니다. 논문 Fig. 2와 Fig. 3의 아키텍처가 직접 코드로 변환된 파일입니다.

4.1 Lévy 노이즈 생성기: Definition 2.1

논문 Definition 2.1 — 대칭 α-stable Lévy 운동

$L^t \sim S^\alpha(1, 0, 0)$ 인 독립적인 대칭 α-stable 확률 변수.

- $\alpha = 2$: 정규분포 (= 브라운 운동)
- $\alpha < 2$: 두꺼운 꼬리(heavy-tailed) 분포 → 큰 점프 허용

Listing 9: networks.py — Lévy 노이즈 생성기 (Definition 2.1)

```python
from scipy.stats import levy_stable

class LevyNoiseGenerator:
    """alpha-stable Levy 노이즈생성기"""

    def __init__(self, alpha=LEVY_ALPHA, beta=LEVY_BETA_PARAM):
        self.alpha = alpha # 1.4 암호화폐(시장)
        self.beta = beta # 0.0 대칭(분포)

    def sample(self, shape):
        samples = levy_stable.rvs(
            alpha=self.alpha, beta=self.beta, size=shape
        )
        samples = np.clip(samples, -5.0, 5.0) # 수치안정성
        return torch.FloatTensor(samples).to(DEVICE)


class GaussianNoiseGenerator:
    """비교용 가우시안노이즈(alpha=2 특수경우)"""
    def sample(self, shape):
        return torch.randn(shape).to(DEVICE)
```

구현 포인트

- `scipy.stats.levy_stable.rvs()`: α-stable 분포에서 직접 샘플링. $\alpha = 1.4, \beta = 0$ (대칭)
- `clip(-5, 5)`: Lévy 분포의 극단값으로 인한 수치 불안정 방지. 논문에서 언급하지 않지만 실용적으로 필수
- `GaussianNoiseGenerator`: 비교 실험(Section 4.3.6, Fig. 9)에서 SDE-DDPG vs SDELP-DDPG 비교 시 사용

4.2 Conv2D 특성 추출기: Fig. 2의 초기 행동 생성

논문 Fig. 2에서 상태 v^t를 Conv2D에 통과시켜 초기 행동 표현 a_1^t를 생성하는 부분입니다.

Listing 10: networks.py — Conv2D 특성 추출기 (논문 Fig. 2 좌측)

```python
class ConvFeatureExtractor(nn.Module):
    """논문
     Fig.2: 로Conv2D 가격텐서에서특성추출입력
    : (batch, 1, n_assets, window) = (B, 1, 9, 50)출력
    : (batch, 128)
    """
    def __init__(self, n_assets, window, out_dim=128):
        super().__init__()
        self.conv_layers = nn.Sequential(
            nn.Conv2d(1, 32, kernel_size=(1, 3), padding=(0, 1)),
            nn.BatchNorm2d(32),
            nn.ReLU(),
            nn.Conv2d(32, 64, kernel_size=(1, 3), padding=(0, 1)),
            nn.BatchNorm2d(64),
            nn.ReLU(),
            nn.Conv2d(64, 64, kernel_size=(1, 3), padding=(0, 1)),
            nn.BatchNorm2d(64),
            nn.ReLU(),
        )
        self.pool = nn.AdaptiveAvgPool2d((n_assets, 1))
        self.fc = nn.Sequential(
            nn.Linear(64 * n_assets, out_dim),
            nn.ReLU(),
        )
```

Conv2D 커널 설계의 의미

`kernel_size=(1, 3)`: 자산 축 방향으로는 크기 1 (자산 간 독립 처리), 시간 축 방향으로는 크기 3 (3일 패턴 감지). 이는 각 자산의 **시계열 지역 패턴**(단기 추세, 변동성 클러스터)을 개별적으로 추출하는 설계입니다. 논문 Table 2의 "가격 시계열의 지역적 패턴 추출"에 해당합니다.

4.3 Drift Net과 Diffusion Net: 식 (7)의 핵심 구성 요소

논문 식 (7) — 행동 정책 생성기 SDE

$$da^t = u(a^t;\theta^u)\,dt + \vartheta(a^t;\theta^\vartheta)\,dL^t_\alpha, \quad a^t \in \mathbb{R}^n$$

- $u(a^t;\theta^u)$: **드리프트 함수** — 행동의 추세적 변화량 (우연적 불확실성 포착)
- $\vartheta(a^t;\theta^\vartheta)$: **확산 함수** — 탐색 범위 제어 (인식론적 불확실성 포착)
- L^t_α: 대칭 α-stable Lévy 과정

Listing 11: networks.py — Drift Net (식 (7)의 u 함수)

```python
class DriftNet(nn.Module):
    """
    Drift 네트워크: u(a_k)논문
    : 만a_k 입력(가state 아님!)
    LayerNorm 사용(은BatchNorm batch에서=1 에러)
    """
    def __init__(self, action_dim, hidden=DRIFT_HIDDEN):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(action_dim, hidden), # 10 -> 64
            nn.LayerNorm(hidden),
            nn.ReLU(),
            nn.Linear(hidden, hidden), # 64 -> 64
            nn.LayerNorm(hidden),
            nn.ReLU(),
            nn.Linear(hidden, action_dim), # 64 -> 10
            nn.Tanh() # 출력범위제한 [-1, 1]
        )
```

Listing 12: networks.py — Diffusion Net (식 (7)의 ϑ 함수)

```python
class DiffusionNet(nn.Module):
    """
    Diffusion 네트워크: theta(a_1)초기행동만
      a_1 입력--- 고정값으로SDE 발산방지
    """
    def __init__(self, action_dim, hidden=DIFFUSION_HIDDEN):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(action_dim, hidden),
            nn.LayerNorm(hidden),
            nn.ReLU(),
            nn.Dropout(0.1),  # 과적합방지
            nn.Linear(hidden, action_dim),
            nn.Softplus()     # 양수보장확산 (계수)
        )
```

LayerNorm vs BatchNorm — 구현의 핵심 판단

논문 Table 2에서는 Drift/Diffusion Net에 "BatchNorm"을 명시하지만, 구현에서는 **LayerNorm**을 사용합니다. 이유:

- 추론(inference) 시 `batch_size=1`로 단일 상태에 대해 행동을 생성
- BatchNorm은 배치 통계량이 필요하므로 `batch=1`에서 에러 발생
- LayerNorm은 단일 샘플 내에서 정규화하므로 배치 크기에 무관
- **ConvFeatureExtractor**와 **Critic**은 BatchNorm2d를 사용 — 이 경우 `agent.py`에서 `eval()`/`train()` 모드 전환으로 대응

4.4 SDE Actor: 식 (8)의 전체 구현

논문 식 (8) — Euler-Maruyama 이산화

$$
\begin{aligned}
a_1^t &= \mathrm{conv2d}(v^t) \\
a_{k+1}^t &= a_k^t + u(a_k^t;\theta^u)\Delta t + \vartheta(a_1^t;\theta^\vartheta)(\Delta t)^{1/\alpha} L_k^t \quad (k = 1, \ldots, m) \\
a^t &= a_{m+1}^t + \mathrm{attention}
\end{aligned}
$$

Listing 13: networks.py — SDEActor.forward() (식 (8) 전체 구현)

```python
def forward(self, state, deterministic=False):
    batch_size = state.shape[0]

    # Step 1: Conv2D 상태특성추출--- a_1 = conv2d(v^t)
    price_tensor, weights = self._split_state(state)
    features = self.conv_extractor(price_tensor) # (batch, 128)

    # Step 2: 초기행동 (a_0)
    init_input = torch.cat([features, weights], dim=-1)
    a_k = self.init_net(init_input)

    # Step 3: SDE 궤적--- K 스텝Euler-Maruyama 식( (8))
    trajectory = [a_k]
    a_first = a_k # 용Diffusion 고정값

    for k in range(self.sde_steps): # m = 5
        # Drift: u(a_k) * dt
        drift = self.drift_net(a_k) * self.dt # dt = 1/5 = 0.2

        if deterministic:
            a_k = a_k + drift # 추론시: 노이즈없음
        else:
            # Diffusion: theta(a_1) * (dt)^{1/alpha} * L_k
            sigma = self.diffusion_net(a_first) # a_1 고정!
            levy_noise = self.levy_noise.sample(
                (batch_size, self.action_dim)
```

```
            )
            noise_scale = self.dt ** (1.0 / self.levy_alpha)
            diffusion = sigma * noise_scale * levy_noise
            a_k = a_k + drift + diffusion

        trajectory.append(a_k)

    # Step 4: Attention --- 최적행동선택
    traj_tensor = torch.stack(trajectory, dim=1) # (B, K+1, action_dim)
    attn_out, _ = self.attention(traj_tensor, traj_tensor, traj_tensor)
    selected = attn_out[:, -1, :]
    selected = self.attention_proj(selected)

    # Step 5: Softmax --- 유효포트폴리오가중치
    action = F.softmax(selected, dim=-1)
    return action
```

식 (8)의 코드 매핑 상세

1. $a_1^t = \textbf{conv2d}(v^t)$: `conv_extractor(price_tensor)` → `init_net()` — 가격 텐서를 Conv2D로 처리하여 128차원 특성 추출 후, 현재 가중치와 결합하여 10차원 초기 행동 생성
2. $a_{k+1} = a_k + u(a_k)\Delta t$: `drift_net(a_k) * self.dt` — Drift Net이 현재 행동 a_k만 입력받아 추세 방향 계산, $\Delta t = 1/m = 0.2$로 스케일링
3. $+\vartheta(a_1)(\Delta t)^{1/\alpha} L_k$: `diffusion_net(a_first) * noise_scale * levy_noise`
   - `a_first`: 초기 행동 a_1을 고정값으로 사용 (SDE 발산 방지)
   - `noise_scale` $= (\Delta t)^{1/\alpha} = 0.2^{1/1.4} \approx 0.305$
   - 이 스케일링은 Lévy 과정의 self-similarity 속성을 반영
4. $a^t = a_{m+1} +$ **attention**: `nn.MultiheadAttention`으로 $K+1$개의 궤적 포인트 중 마지막 시점의 가중 합산 출력 사용
5. **Softmax**: 포트폴리오 가중치의 합 $= 1$ 제약 보장

deterministic 모드의 의미

`deterministic=True`일 때 Lévy 노이즈와 확산 항이 비활성화됩니다:

- **학습 중**: `deterministic=False` — SDE 내부 노이즈로 다양한 궤적 탐색
- **Actor 업데이트 시**: `deterministic=True` — DPG(식 (14))는 결정적 정책의 그래디언트를 필요로 함
- **테스트/추론 시**: 탐색이 불필요하므로 결정적 행동 생성

이는 논문 식 (16)에서 SDE 출력 a에 별도의 OU 노이즈를 추가하는 구조와 일치합니다 — SDE 내부 노이즈는 "정책의 확률적 모델링"이고, OU 노이즈는 "학습 중 추가 탐색"입니다.

4.5 Critic 네트워크: 논문 Fig. 3

논문 Fig. 3 — 다층 합성곱 잔차 신경망

Critic은 State 경로(Conv2D + Residual Block × 3)와 Action 경로(Dense)를 분리 처리한 후 병합하여 Q-value를 출력합니다.

Listing 14: networks.py — Residual Block (논문 Fig. 3 내부)

```python
class ResidualBlock(nn.Module):
    """논문 Fig.3: Conv2D -> ReLU -> Conv2D -> Add(Skip) -> ReLU"""
    def __init__(self, channels):
        super().__init__()
        self.block = nn.Sequential(
            nn.Conv2d(channels, channels, kernel_size=(1, 3), padding=(0, 1)),
            nn.BatchNorm2d(channels),
            nn.ReLU(),
            nn.Conv2d(channels, channels, kernel_size=(1, 3), padding=(0, 1)),
            nn.BatchNorm2d(channels),
        )

    def forward(self, x):
        return F.relu(self.block(x) + x) # Skip connection
```

Listing 15: networks.py — Critic (논문 Fig. 3 전체)

```python
class Critic(nn.Module):
    """
    State 경로: Conv2D + ReLU -> ResBlock x 3 -> Conv2D -> Pool -> Dense
    Action 경로: Dense -> ReLU
    Merge: concat(State, Action) -> Dense -> Dense -> Q-value 차원(1)
    """
    def __init__(self, state_dim, action_dim, hidden=128, num_res_blocks=3):
        super().__init__()
        # State 경로: Conv2D + Residual
        self.state_conv = nn.Sequential(
            nn.Conv2d(1, 32, kernel_size=(1, 3), padding=(0, 1)),
            nn.BatchNorm2d(32), nn.ReLU(),
        )
        self.res_blocks = nn.Sequential(
            *[ResidualBlock(32) for _ in range(num_res_blocks)]
        )
        # ...
        # Merge -> Q-value
        self.merge = nn.Sequential(
            nn.Linear(hidden * 2, hidden), nn.ReLU(),
            nn.Linear(hidden, hidden), nn.ReLU(),
            nn.Linear(hidden, 1), # Q-value 스칼라출력
        )
```

Critic의 역할 — 논문 전체에서의 위치

Critic은 $Q^{\Phi}(v, a)$를 근사합니다 (식 (10)). 이 Q값은:

- **Critic 학습** (식 (11)/(17)): TD 오차를 MSE로 최소화
- **Actor 학습** (식 (15)): $\nabla_a Q$가 Actor의 정책 그래디언트 방향을 결정
- 즉, Critic이 "현재 행동을 어떻게 바꾸면 Q값이 올라가는가"를 알려주면, Actor는 그 방향으로 drift/diffusion 네트워크를 업데이트

5 DDPG 에이전트: `agent.py` (식 (10)∼(17), Algorithm 1)

`agent.py`는 논문 Algorithm 1의 전체 학습 알고리즘을 구현합니다. Session 2에서 분석한 식 (10)∼(17)이 하나의 클래스에 통합됩니다.

5.1 Ornstein-Uhlenbeck 탐색 노이즈: 식 (16)

논문 식 (16) — OU 프로세스 + 정규화

$$a^* = \frac{a + \mathfrak{N}}{\sum_j a_j}, \quad j = 1, \ldots, n$$

$\mathfrak{N}$은 Ornstein-Uhlenbeck 프로세스의 노이즈입니다.

Listing 16: agent.py — OU Process (논문 식 (16)의 $\mathfrak{N}$)

```python
class OUProcess:
    """
    Ornstein-Uhlenbeck Process 탐색( 노이즈)
    dX_t = theta(mu - X_t)dt + sigma * dW_t평균회귀특성
      -> 모멘텀있는탐색
    """
    def __init__(self, size, theta=0.15, sigma=0.2, mu=0.0):
        self.size = size
        self.theta = theta # 평균회귀속도
        self.sigma = sigma # 노이즈강도
        self.mu = mu * np.ones(size)
        self.reset()

    def sample(self):
        dx = self.theta * (self.mu - self.state) + \
             self.sigma * np.random.randn(self.size)
        self.state += dx
        return self.state.copy()
```

SDE 내부 Lévy 노이즈 vs OU 프로세스 — 이중 노이즈 구조

Session 2에서 분석한 식 (16)의 핵심 포인트:

- **SDE 내부 Lévy 노이즈**: 정책 자체의 "확률적 동역학 모델링" — 학습 완료 후에도 정책의 본질적 구성 요소
- **OU 프로세스 노이즈**: "학습 중 추가 탐색"을 위한 것 — 테스트 시 제거 가능
- OU의 **평균 회귀** 성질: 랜덤 워크처럼 무한정 발산하지 않고 평균으로 돌아옴 → 극단적 비중 변화(큰 거래 비용) 방지

5.2 경험 리플레이 버퍼: Algorithm 1 Line 14

Listing 17: agent.py — Replay Buffer (Algorithm 1 Line 3, 14)

```python
class ReplayBuffer:
    """경험 리플레이버퍼"""
    def __init__(self, capacity=50000):
        self.buffer = deque(maxlen=capacity)

    def push(self, state, action, reward, next_state, done):
        self.buffer.append((state, action, reward, next_state, done))

    def sample(self, batch_size=64):
        batch = random.sample(self.buffer, batch_size)
        states, actions, rewards, next_states, dones = zip(*batch)
        return (
            torch.FloatTensor(np.array(states)).to(DEVICE),
            # ... 텐서변환및 GPU 전송
        )
```

리플레이 버퍼의 역할 — 식 (10)

식 (10)과 (11)에서 $\mathbb{E}_{v' \sim \mathcal{D}}$의 $\mathcal{D}$가 바로 이 리플레이 버퍼입니다:

- 금융 시계열은 **자기상관**이 강함 — 순차적 학습 시 연속된 샘플 간 상관관계로 학습 불안정
- 무작위 샘플링으로 시간적 상관관계를 깨뜨려 i.i.d. 가정에 근접
- `deque(maxlen=50000)`: 최근 50,000개 경험만 유지 (오래된 비정상 데이터 자동 폐기)

5.3 에이전트 초기화: Algorithm 1 Line 1∼3

Listing 18: agent.py — 에이전트 초기화 (Algorithm 1 Line 1∼3)

```python
class SDELPDDPGAgent:
    def __init__(self, state_dim, action_dim):
        # Line 1: Actor (Online + Target)
        self.actor = SDEActor(state_dim, action_dim).to(DEVICE)
        self.actor_target = SDEActor(state_dim, action_dim).to(DEVICE)
        self.actor_target.load_state_dict(self.actor.state_dict())
        self.actor_target.eval() # 항상eval 모드
        self.actor_optimizer = optim.Adam(
            self.actor.parameters(), lr=ACTOR_LR
        )
        # Line 2: Critic (Online + Target)
        self.critic = Critic(state_dim, action_dim).to(DEVICE)
        self.critic_target = Critic(state_dim, action_dim).to(DEVICE)
        self.critic_target.load_state_dict(self.critic.state_dict())
        self.critic_target.eval()
        self.critic_optimizer = optim.Adam(
            self.critic.parameters(), lr=CRITIC_LR
        )
        # Line 3: 리플레이버퍼 + OU 프로세스
        self.replay_buffer = ReplayBuffer()
        self.ou_process = OUProcess(action_dim)
```

6개 네트워크 구조 — 논문 Section 3.3.5

논문에서는 Actor가 Drift-Net과 Diffusion-Net으로 분리되어 총 **6개** 네트워크가 필요합니다. 구현에서는 SDEActor가 내부에 drift_net, diffusion_net을 포함하므로:

- `self.actor` = Online Actor (drift_net + diffusion_net + init_net + attention 포함)
- `self.actor_target` = Target Actor (동일 구조)
- `self.critic`, `self.critic_target` = Online/Target Critic

소프트 업데이트 시 Actor 전체가 한 번에 업데이트되므로, drift_net과 diffusion_net의 파라미터가 동시에 갱신됩니다.

5.4 행동 선택: Algorithm 1 Line 7∼12 (식 (8) + 식 (16))

Listing 19: agent.py — 행동 선택 (식 (8) + 식 (16))

```python
def select_action(self, state, explore=True):
    state_tensor = torch.FloatTensor(state).unsqueeze(0).to(DEVICE)

    self.actor.eval() # BatchNorm eval 모드(batch=1 대응)
    with torch.no_grad():
        # 식 (8): SDE Actor 순전파
        action = self.actor(state_tensor, deterministic=not explore)
```

```
        action = action.cpu().numpy()[0]
    self.actor.train() # 학습모드복원

    if explore:
        # 식 (16): OU 프로세스노이즈추가
        noise = self.ou_process.sample() * 0.1
        action = action + noise
        # 음수방지후재정규화포트폴리오 ( 가중치합 = 1)
        action = np.clip(action, 0.0, None)
        action_sum = action.sum()
        if action_sum > 0:
            action = action / action_sum # sum_j 로a_j 나누기
        else:
            action = np.ones(self.action_dim) / self.action_dim
    return action
```

식 (16)의 코드 매핑

$$a^* = \frac{a + \mathfrak{N}}{\sum_j a_j}$$

1. a: `self.actor(state_tensor)` — SDE Actor의 출력 (식 (8))
2. $\mathfrak{N}$: `self.ou_process.sample() * 0.1` — OU 노이즈 (0.1 스케일)
3. $a + \mathfrak{N}$: `action + noise`
4. $\sum_j a_j$로 나누기: `action / action_sum` — 포트폴리오 가중치 합 = 1 보장

`explore=False`일 때는 OU 노이즈를 추가하지 않고 Actor의 결정적 출력만 사용합니다 (테스트/백테스트 시).

5.5 학습 업데이트: 식 (15) + 식 (17) — Algorithm 1의 핵심

논문 식 (15) — SDELP-DDPG의 핵심 (체인 룰 분해)

$$\nabla_\theta J = \mathbb{E}_{v\sim\mathcal{D}}\Big\{\nabla_a Q^\Phi(v, a \mid \theta^Q)\Big|_{a=\Phi(v,L|\theta^u,\theta^\vartheta)} \cdot \big[\nabla_{\theta^u}\Phi(\cdot) + \nabla_{\theta^\vartheta}\Phi(\cdot)\big]\Big\}$$

Critic의 $\nabla_a Q$가 Actor 내부에서 drift/diffusion 두 갈래로 분기됩니다.

논문 식 (17) — Critic 업데이트

$$\mathcal{L}(\theta^Q) = \mathbb{E}_{v,a,R,v'}\Big[\big(Q(v,a) - y\big)^2\Big]$$

$$y = R^t + \gamma \max_{a'} \tilde{Q}(v', a'^*), \quad a' = \Phi(v', L \mid \theta^u, \theta^\vartheta), \quad a'^* = \frac{a' + \mathfrak{N}}{\sum_j a'_j}$$

Listing 20: agent.py — update() 메서드 (식 (15) + (17) 구현)

```python
def update(self):
    if len(self.replay_buffer) < BATCH_SIZE:
        return None, None

    # 미니배치샘플링 (Algorithm 1 Line 15-16)
    states, actions, rewards, next_states, dones = \
        self.replay_buffer.sample()

    #   Critic 업데이트: 식(17)
    with torch.no_grad():
        next_actions = self.actor_target(
            next_states, deterministic=True
        )
        target_q = self.critic_target(next_states, next_actions)
        # TD Target: y = R + gamma * Q'(s', a')
        y = rewards + GAMMA * target_q * (1 - dones)

    current_q = self.critic(states, actions)
    # Critic Loss: MSE --- 식(17)의 L(theta^Q)
    critic_loss = nn.MSELoss()(current_q, y)

    self.critic_optimizer.zero_grad()
    critic_loss.backward()
    torch.nn.utils.clip_grad_norm_(self.critic.parameters(), 1.0)
    self.critic_optimizer.step()

    #   Actor 업데이트: 식(15)
    # DPG: -E[Q(s, pi(s))] (를Q 최대화하는방향으로)
    predicted_actions = self.actor(states, deterministic=True)
    actor_loss = -self.critic(states, predicted_actions).mean()
    self.actor_optimizer.zero_grad()
    actor_loss.backward() # 자동미분으로식(15)의 체인룰계산
    torch.nn.utils.clip_grad_norm_(self.actor.parameters(), 1.0)
    self.actor_optimizer.step()

    #   Target 소프트업데이트: Algorithm 1 Line 20
    self._soft_update(self.actor, self.actor_target)
    self._soft_update(self.critic, self.critic_target)
    return critic_loss.item(), actor_loss.item()
```

5.6 소프트 타겟 업데이트: Algorithm 1 Line 20

Listing 21: agent.py — 소프트 업데이트 (Algorithm 1 Line 20)

```python
def _soft_update(self, source, target):
    """theta' <- tau * theta + (1 - tau) * theta'"""
    for src_param, tgt_param in zip(
        source.parameters(), target.parameters()
    ):
        tgt_param.data.copy_(
            TAU * src_param.data + (1 - TAU) * tgt_param.data
        )
        # TAU = 0.001: 매번만0.1% 교체(EMA)
```

소프트 업데이트의 의미

Session 2에서 분석한 "안정화 기법 1: Target Q-network"의 구현입니다:

- $\varphi = 0.001$ (= `TAU`): 매 업데이트마다 Target이 Main의 0.1%만 흡수
- 목표값 y가 급격히 변하지 않아 "움직이는 목표(moving target)" 문제 완화
- Actor와 Critic 모두에 적용 (논문: Online/Target Drift-Net, Diffusion-Net, Critic 각각)

식 (15)가 PyTorch 자동미분으로 구현되는 원리

식 (15)의 체인 룰: $\frac{\partial J}{\partial \theta^u} = \frac{\partial Q}{\partial a} \cdot \frac{\partial \Phi}{\partial \theta^u}, \quad \frac{\partial J}{\partial \theta^\vartheta} = \frac{\partial Q}{\partial a} \cdot \frac{\partial \Phi}{\partial \theta^\vartheta}$
PyTorch에서는 다음 두 줄로 자동 계산됩니다:

1. `predicted_actions = self.actor(states)` — 순전파 그래프 구성
2. `actor_loss = -self.critic(states, predicted_actions).mean()` — Q값 계산
3. `actor_loss.backward()` — **자동미분**이 Critic → Actor → drift_net/diffusion_net 순으로 그래디언트를 역전파

즉, `backward()`가 호출되면:

- $\nabla_a Q$: Critic 내부에서 action에 대한 그래디언트 자동 계산
- $\nabla_{\theta^u} \Phi$: drift_net의 파라미터에 대한 그래디언트 자동 계산
- $\nabla_{\theta^\vartheta} \Phi$: diffusion_net의 파라미터에 대한 그래디언트 자동 계산
- 이 세 그래디언트의 곱이 체인 룰에 의해 자동으로 합산됩니다

6 학습 루프: train.py (Algorithm 1 외부 루프)

train.py는 Algorithm 1의 for epoch = 1 to N 외부 루프를 구현하며, 환경 초기화, 에이전트 상호작용, 학습 진행, 결과 시각화를 관리합니다.

6.1 학습 메인 루프: Algorithm 1 Line 4∼21

Listing 22: train.py — 학습 메인 루프 (Algorithm 1 전체)

```python
def train():
    # Algorithm 1 Line 1-3: 환경에이전트/ 초기화
    env = PortfolioEnv()
    agent = SDELPDDPGAgent(env.state_dim, env.action_dim)

    episode_rewards = []
    episode_values = []
    all_portfolio_values = []
    best_value = 0.0

    for ep in range(NUM_EPISODES): # Line 4: for epoch = 1 to N
        state = env.reset()      # Line 5: 초기관측 v^0
        agent.ou_process.reset() # Line 5: OU 프로세스초기화
        ep_reward = 0.0

        while True:              # Line 6: for t = 1 to T
            # Line 7-12: 행동생성 (SDE + OU)
            action = agent.select_action(state, explore=True)

            # Line 13: 환경실행, 보상관측
            next_state, reward, done, info = env.step(action)

            # Line 14: 경험저장
            agent.replay_buffer.push(
                state, action, reward, next_state, float(done)
            )

            # Line 15-20: 네트워크업데이트
            agent.update()

            ep_reward += reward
            state = next_state
            if done:
                break

        final_value = info.get("portfolio_value", env.portfolio_value)
        episode_rewards.append(ep_reward)
        episode_values.append(final_value)
        all_portfolio_values.append(env.portfolio_values.copy())

        # 최고성과모델저장
        if final_value > best_value:
            best_value = final_value
```

```
44            agent.save(os.path.join(OUTPUT_DIR, "best_model.pt"))
45
46        # 결과시각화
47        plot_results(episode_rewards, episode_values,
48                     all_portfolio_values, env, agent)
49        return agent, env, episode_rewards, episode_values
```

Algorithm 1과 코드의 1:1 대응

Algorithm 1	코드	수식
Line 4: for epoch	`for ep in range(NUM_EPISODES)`	—
Line 5: 초기 관측	`env.reset()`	$a^0 = (1, 0, \ldots, 0)$
Line 7-10: SDE 순전파	`agent.actor(state)`	식 (8)
Line 11-12: OU + 정규화	`ou_process.sample()`	식 (16)
Line 13: 환경 실행	`env.step(action)`	식 (1)∼(6)
Line 14: 버퍼 저장	`replay_buffer.push()`	$\mathcal{D}$
Line 17: Critic 업데이트	`critic_loss.backward()`	식 (17)
Line 18: Actor 업데이트	`actor_loss.backward()`	식 (15)
Line 20: 소프트 업데이트	`_soft_update()`	$\varphi = 0.001$

6.2 모델 저장

Listing 23: agent.py — 모델 저장/로드

```
1 def save(self, path):
2     torch.save({
3         "actor": self.actor.state_dict(),
4         "critic": self.critic.state_dict(),
5         "actor_target": self.actor_target.state_dict(),
6         "critic_target": self.critic_target.state_dict(),
7     }, path)
```

`best_model.pt`는 학습 중 **최고 포트폴리오 가치**를 달성한 시점의 가중치를 저장합니다. 이 모델이 백테스트에서 사용됩니다.

7 백테스트: `backtest.py` (식 (19), Table 2, Table 3)

`backtest.py`는 학습된 모델을 테스트 기간(2023-01-01 ∼ 2026-01-31)에 적용하여 논문 Table 2, Table 3의 평가지표를 산출합니다.

7.1 평가지표 계산: 논문 식 (19)

논문 식 (19) — 성과 평가지표

$$\text{Annual Return} = \Big((1 + R_E)^{365/y} - 1.0\Big) \times 100\%$$

$$\text{Sharpe Ratio} = \frac{E[R_s - R_f]}{\sigma}$$

Listing 24: backtest.py — 평가지표 계산 (논문 식 (19), Table 3)

```python
def compute_metrics(portfolio_values):
    """논문 Table 의3 평가지표계산"""
    values = np.array(portfolio_values)
    T = len(values) - 1
    years = T / 252 # 연간거래일

    daily_returns = values[1:] / values[:-1] - 1

    # CRR (Cumulative Return Ratio)
    crr = values[-1] / values[0]

    # AR (Annualized Return) --- 식(19)
    ar = (crr ** (1 / years) - 1) * 100

    # AV (Annualized Volatility)
    av = np.std(daily_returns) * np.sqrt(252) * 100

    # Sharpe Ratio --- 식(19)
    mean_daily = np.mean(daily_returns)
    std_daily = np.std(daily_returns)
    sharpe = (mean_daily / std_daily) * np.sqrt(252) \
            if std_daily > 0 else 0

    # Sortino Ratio 하방(위험만고려)
    downside = daily_returns[daily_returns < 0]
    downside_std = np.std(downside) \
                if len(downside) > 0 else 1e-8
    sortino = (mean_daily / downside_std) * np.sqrt(252)

    # MDD (Maximum Drawdown)
    peak = np.maximum.accumulate(values)
    drawdown = (peak - values) / peak
    mdd = np.max(drawdown) * 100

    return {"CRR": crr, "AR (%)": ar, "AV (%)": av,
           "Sharpe": sharpe, "Sortino": sortino, "MDD (%)": mdd}
```

각 지표의 의미

지표	의미	좋은 방향
CRR	누적 수익 배수 (P_T/P_0)	높을수록 좋음
AR (%)	연간 환산 수익률	높을수록 좋음
AV (%)	연간 환산 변동성	낮을수록 좋음
Sharpe	위험 대비 초과 수익률	높을수록 좋음 (> 1 양호)
Sortino	하방 위험 대비 수익률	높을수록 좋음
MDD (%)	최대 낙폭	낮을수록 좋음

논문 Table 2에서 SDELP-DDPG는 AR, Sharpe, Sortino, CRR에서 최고 성능을 보이지만,

AV와 MDD에서는 뚜렷한 우위가 없습니다 — 이는 보상 함수 식 (6)이 변동성과 낙폭을 직접 다루지 않기 때문입니다.

7.2 백테스팅 실행 흐름

Listing 25: backtest.py — 백테스팅 루프

```python
def backtest():
    # 테스트환경 (2023-01-01 ~ 2026-01-31)
    test_env = PortfolioEnv(
        tickers=TICKERS, start=TEST_START, end=TEST_END
    )

    # 학습된모델로드
    agent = SDELPDDPGAgent(test_env.state_dim, test_env.action_dim)
    agent.load("outputs/best_model.pt")

    # 탐색없이결정적행동만사용
    state = test_env.reset()
    while True:
        action = agent.select_action(state, explore=False) # No OU noise
        next_state, reward, done, info = test_env.step(action)
        state = next_state
        if done:
            break

    # 성과비교 : SDELP-DDPG vs Buy-and-Hold
    sdelp_metrics = compute_metrics(test_env.portfolio_values)
    bah_metrics = compute_metrics(test_env.get_buy_and_hold_values())
```

explore=False의 의미

백테스트에서는 `explore=False`로 설정합니다:

- OU 프로세스 노이즈 비활성화
- SDE Actor는 `deterministic=True`로 동작 — Lévy 노이즈 없이 drift만 사용
- 즉, 학습된 drift net이 "시장 상황에 따른 최적 재배분 방향"을 순수하게 결정

8 Lévy vs Gaussian 비교: compare_levy_gaussian.py

이 모듈은 논문 Section 4.3.5 (-stable of Lévy)와 Section 4.3.6 (Fig. 9)의 실험적 근거를 시각적으로 제공합니다.

8.1 노이즈 분포 비교: 논문 Table 1의 시각화

Listing 26: compare_levy_gaussian.py — 분포 비교 (Table 1 시각화)

```
def compare_noise_distributions():
    n_samples = 10000

    # 가우시안 (alpha=2 특수경우)
    gaussian_samples = np.random.randn(n_samples)

    # Levy (alpha=1.5, alpha=1.8)
    levy_samples_15 = levy_stable.rvs(alpha=1.5, beta=0, size=n_samples)
    levy_samples_15 = np.clip(levy_samples_15, -10, 10)

    levy_samples_18 = levy_stable.rvs(alpha=1.8, beta=0, size=n_samples)
    levy_samples_18 = np.clip(levy_samples_18, -10, 10)

    # 4-panel 시각화:
    # (1) 히스토그램비교 --- fat tail 확인
    # (2) 로그스케일 tail 비교--- 꼬리두께정량화
    # (3) 시계열비교 --- 탐색궤적의점프특성
    # (4) 별alpha 바이올린플롯--- 탐색범위비교
```

8.2 행동 분포 비교

Listing 27: compare_levy_gaussian.py — 행동 분포 비교

```
def compare_action_generation():
    """SDE 에서Actor Levy vs Gaussian 행동분포비교"""
    drift = np.array([0.2, 0.5, 0.3]) # 가상drift (BTC 선호)

    # Gaussian 행동: drift + N(0, 0.3^2) -> softmax
    # Levy 행동: drift + Levy(1.5) * 0.3 -> softmax

    # 결과: 가Levy 더넓은행동분포  -> 다양한포트폴리오탐색
```

비교 결과의 논문적 의미

이 비교는 논문의 핵심 주장을 시각적으로 입증합니다:

- **Gaussian** ($\alpha = 2$): 좁은 범위의 점진적 탐색 — 지역 최적에 빠질 위험
- **Lévy** ($\alpha < 2$): 간헐적 큰 점프로 넓은 탐색 — 지역 최적 탈출 가능
- 논문 Fig. 9에서 SDELP-DDPG의 loss가 SDE-DDPG보다 빠르게 안정화되는 것이 이를 뒷받침

9 통합 실행: run_all.py

Listing 28: run_all.py — 전체 파이프라인

```python
def main():
    # Step 1: Levy vs Gaussian 비교시각화
    from compare_levy_gaussian import (
        compare_noise_distributions, compare_action_generation
    )
    compare_noise_distributions() # -> outputs/levy_vs_gaussian.png
    compare_action_generation() # -> outputs/action_distribution.png

    # Step 2: SDELP-DDPG 학습
    from train import train
    agent, env, rewards, values = train()
    # -> outputs/training_results.png
    # -> outputs/losses.png
    # -> outputs/best_model.pt

    # 최종: 생성된파일목록출력
```

주의: 백테스트는 포함되지 않음

run_all.py는 노이즈 비교 + 학습만 실행합니다. 백테스트(backtest.py)는 학습 완료 후 별도로 실행해야 합니다:

```
python run_all.py          # Step 1: 비교 시각화 + 학습
python backtest.py         # Step 2: 테스트 기간 평가
```

10 출력 결과 해석

outputs/ 폴더에 생성되는 시각화 결과물을 논문의 맥락에서 상세히 해석합니다.

10.1 Figure 1: levy_vs_gaussian.png — Lévy vs Gaussian 노이즈 비교

이 그래프는 4개의 서브플롯으로 구성됩니다.

Panel ①: Distribution Comparison (히스토그램)

Gaussian($\alpha = 2$)과 Lévy($\alpha = 1.5$)의 확률밀도함수를 오버레이합니다.

- **관찰**: Lévy 분포가 중심(peak)이 더 뾰족하고 꼬리(tail)가 더 두꺼움
- **의미**: 대부분의 시간에는 작은 변화(보수적), 가끔 큰 점프(적극적 탐색)
- **논문 연결**: 이것이 금융 시장의 "대부분 안정 + 간헐적 급변" 특성을 포착하는 핵심 메커니즘 (논문 Section 2.3)

Panel ②: Log-Scale Tail Comparison

로그 스케일로 꼬리 확률을 비교합니다.

- **관찰**: $|x| > 3$ 영역에서 Lévy의 확률밀도가 Gaussian보다 수십~수백 배 높음
- **의미**: Lévy는 "극단적 이벤트"의 확률이 Gaussian보다 훨씬 높음
- **논문 연결**: 논문 Table 1의 Kurtosis(첨도)가 α가 작을수록 높아지는 것과 일치 — 두꺼운 꼬리의 정량적 확인

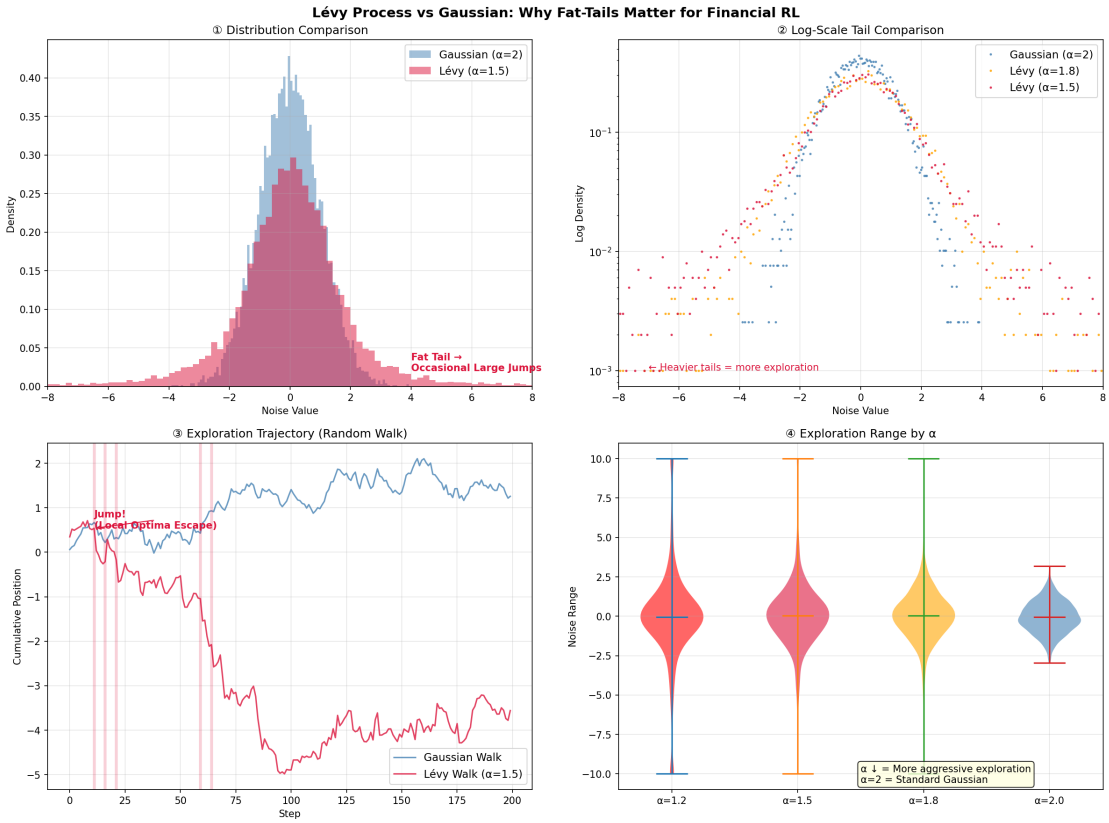

Figure 1: Lévy vs Gaussian 노이즈 비교 — 4개 패널로 두 분포의 특성 차이를 시각화

Panel ③: Exploration Trajectory (Random Walk)

200스텝 누적 합(random walk)을 비교합니다.

- **관찰**: Gaussian walk은 부드러운 곡선, Lévy walk은 간헐적 "점프"가 보임
- **의미**: Lévy 기반 탐색은 현재 위치에서 멀리 떨어진 행동을 시도 가능 — **지역 최적 탈출**
- **논문 연결**: Session 2의 "핵심 문제 인식" — 기존 DRL의 탐색 전략은 보상 값과 무관하게 무작위로 선택하므로 넓은 범위 탐색이 불가능. Lévy가 이를 해결

Panel ④: Exploration Range by α (바이올린 플롯)

$\alpha = 1.2, 1.5, 1.8, 2.0$에 대한 노이즈 분포 범위를 비교합니다.

- **관찰**: α가 작을수록 분포의 "수염"이 길어짐 (극단값 빈도 증가)
- **의미**: α는 탐색의 "공격성"을 제어하는 하이퍼파라미터
- **논문 연결**: 논문 Table 1에서 DJIA(안정적 시장) $\alpha = 1.7$, Energy(변동성 큰 시장) $\alpha = 1.4$가 최적 — 변동성 큰 시장일수록 더 적극적 탐색 필요

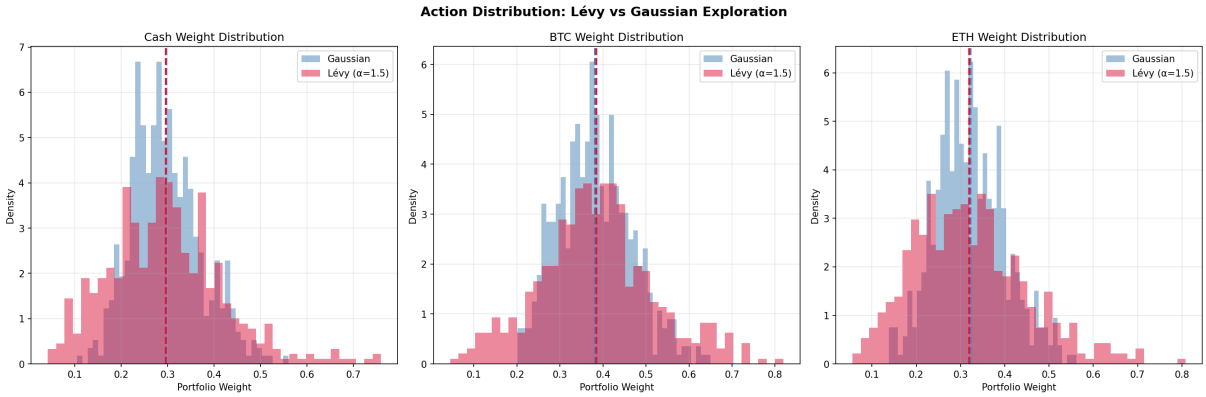

Figure 2: 행동 분포 비교 — Gaussian vs Lévy 기반 포트폴리오 가중치 분포 (Cash, BTC, ETH)

10.2 Figure 2: `action_distribution.png` — 행동 분포 비교

3개의 서브플롯(Cash, BTC, ETH)으로 Gaussian vs Lévy 기반 포트폴리오 가중치 분포를 비교합니다.

해석

- **관찰**: Lévy 기반 행동이 더 넓은 가중치 범위를 탐색 — 특히 극단적 배분(0에 가깝거나 높은 비중)을 더 자주 시도
- **의미**: 학습 초기에 다양한 포트폴리오 전략을 시도하여 최적 전략 발견 확률 증가
- **논문 연결**: 식 (8)의 확산 항 $\vartheta(a_1)(\Delta t)^{1/\alpha}L_k$가 행동 공간에서 더 넓은 영역을 커버함을 직접 확인

10.3 Figure 3: `training_results.png` — 학습 결과 4-Panel

Panel ①: Learning Curve (에피소드별 누적 보상)

- **X축**: 에피소드, **Y축**: 에피소드 내 누적 보상 $\sum_t R_t$
- **관찰 포인트**: 보상이 에피소드가 진행될수록 증가 추세인지, 수렴하는지 확인
- **의미**: 에이전트가 식 (6)의 보상(수익 - 리스크 페널티)을 점점 더 잘 최대화하고 있음
- **논문 연결**: 논문 Fig. 7에서 보상이 에피소드에 따라 증가하며, $\beta = 0.05$에서 최대값 달성
- **이동평균선**(10 에피소드): 노이즈가 많은 원시 보상의 추세를 확인하는 데 활용

Panel ②: Portfolio Value vs Buy & Hold (마지막 에피소드)

- **빨간선**: SDELP-DDPG 에이전트의 포트폴리오 가치 곡선
- **회색 점선**: 동일 비중 Buy & Hold 벤치마크
- **검은 점선**: $P_t/P_0 = 1.0$ (손익분기)
- **관찰 포인트**: 에이전트가 벤치마크 대비 얼마나 초과 수익을 달성했는지, 하락장에서 방어력은 어떤지 확인
- **논문 연결**: 논문 Fig. 8의 APV(Accumulated Portfolio Value) 그래프에 해당. SDELP-

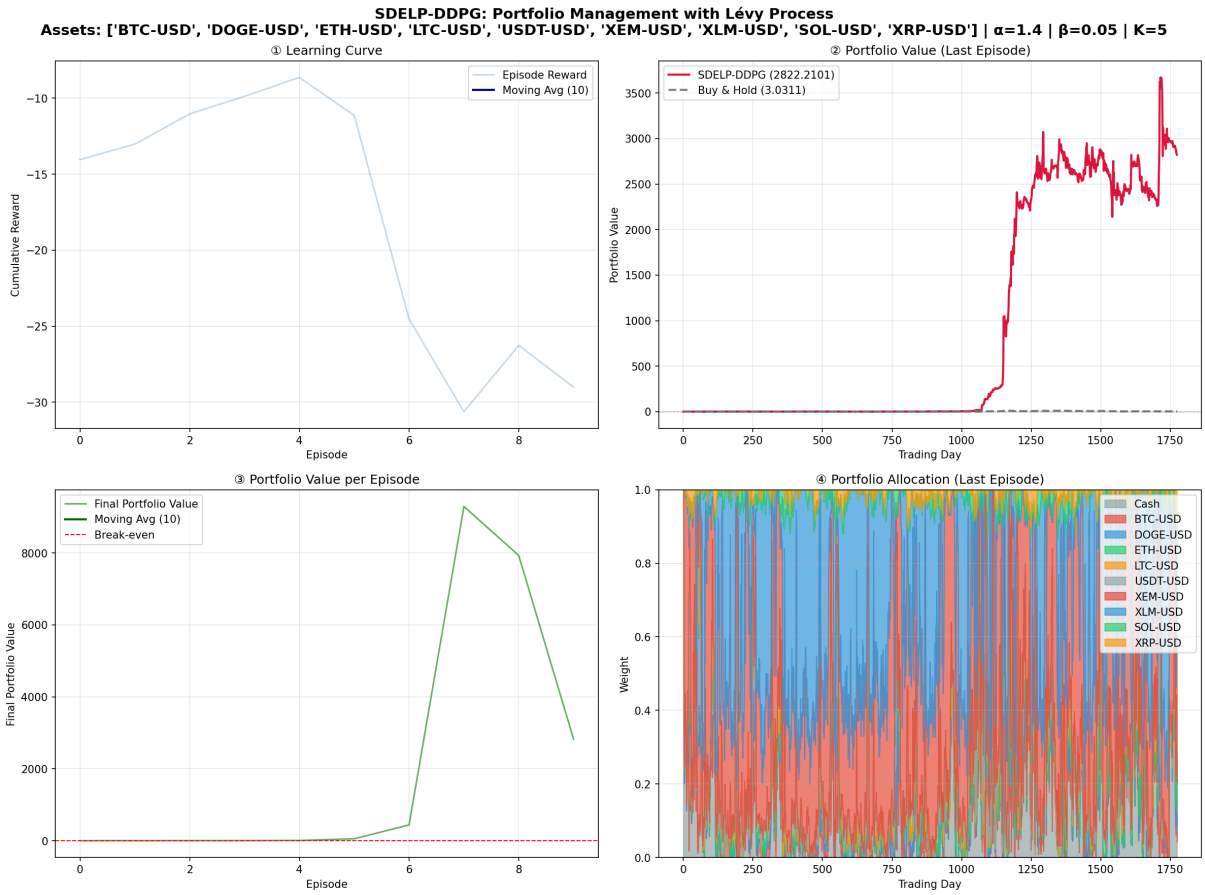

Figure 3: 학습 결과 4-Panel — Learning Curve, Portfolio Value, Episode별 가치, 자산 배분 변화

DDPG가 "전체 기간에 걸쳐 일관되게 상위" 성과를 보여야 함

Panel ③: Portfolio Value per Episode

- **X축**: 에피소드, **Y축**: 에피소드 종료 시 포트폴리오 가치 (P_T/P_0)
- **빨간 점선**: 손익분기 ($P_T = P_0$, 즉 가치 = 1.0)
- **관찰 포인트**: 에피소드가 진행될수록 포트폴리오 가치가 1.0 이상으로 안정화되는지 확인
- **논문 연결**: 학습 수렴의 지표. 가치가 1.0 미만이면 에이전트가 아직 수익을 내지 못하는 상태

Panel ④: Portfolio Allocation (가중치 변화)

- **적층 영역 그래프**: 각 자산(Cash, BTC, ETH, ⋯)의 일별 포트폴리오 비중 변화
- **관찰 포인트**: 에이전트가 시장 상황에 따라 동적으로 자산 배분을 조정하는지, 특정 자산에 과도하게 집중하지 않는지 확인
- **논문 연결**: 식 (6)의 KL 페널티($\beta \cdot D$)가 작동하면 가중치 변화가 부드럽게 유지됨. 급격한 변화는 큰 거래 비용(식 (3))을 유발하므로 에이전트가 이를 학습적으로 회피

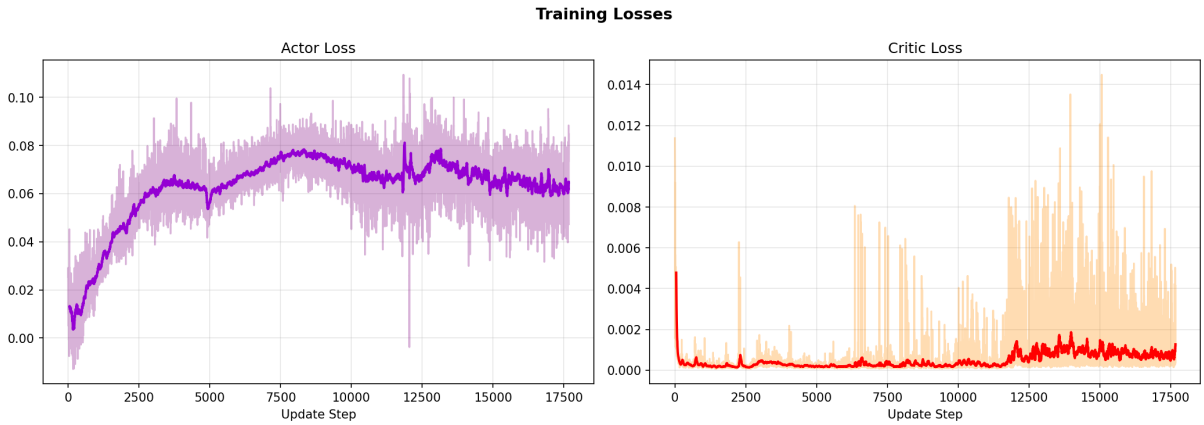

Figure 4: Actor/Critic Loss 추이 — 좌: Actor Loss ($-\mathbb{E}[Q]$), 우: Critic Loss (TD 오차 MSE)

- **이상적 패턴**: Cash 비중이 하락장에서 증가하고, 상승장에서 암호화폐 비중이 증가하는 적응적 배분

10.4 Figure 4: `losses.png` — Actor/Critic Loss

좌측: Actor Loss

- **정의**: $-\mathbb{E}[Q(s, \pi_\theta(s))]$ — Q값의 음수 (최소화 = Q값 최대화)
- **이상적 패턴** (논문): 점진적으로 감소 후 안정화
- **실제 관찰**: 초반 감소 후 재상승하는 U자형 곡선 — 10 에피소드의 짧은 학습으로 Actor가 충분히 수렴하지 못함을 시사
- **논문 연결**: 식 (15)의 정책 그래디언트 $\nabla_\theta J$에 의한 Actor 업데이트 결과. 논문 Fig. 9 우측의 "actor loss of two markets"에 대응
- **해석**: Loss 값이 감소하면 ($-\mathbb{E}[Q]$가 작아지면) Q값이 증가하고 있음을 의미. 본 실험에서는 Q값이 음수 영역이므로 Loss가 양수로 나타남

우측: Critic Loss

- **정의**: $\mathbb{E}[(Q(v, a) - y)^2]$ — TD 오차의 MSE
- **이상적 패턴** (논문): 빠르게 감소 후 낮은 수준에서 안정화
- **실제 관찰**: 대체로 낮은 수준(0.001~0.004)을 유지하나 후반부에 간헐적 스파이크 발생 — 학습 후반 비정상적 상태 진입 시 TD 오차 급증
- **논문 연결**: 식 (17)의 $\mathcal{L}(\theta^Q)$. 논문 Fig. 9 좌측의 "critic loss of two markets"에 대응. 논문에서는 46 에포크 후 안정화를 보고하나, 본 실험(10 에피소드)에서는 완전한 안정화에 도달하지 못함

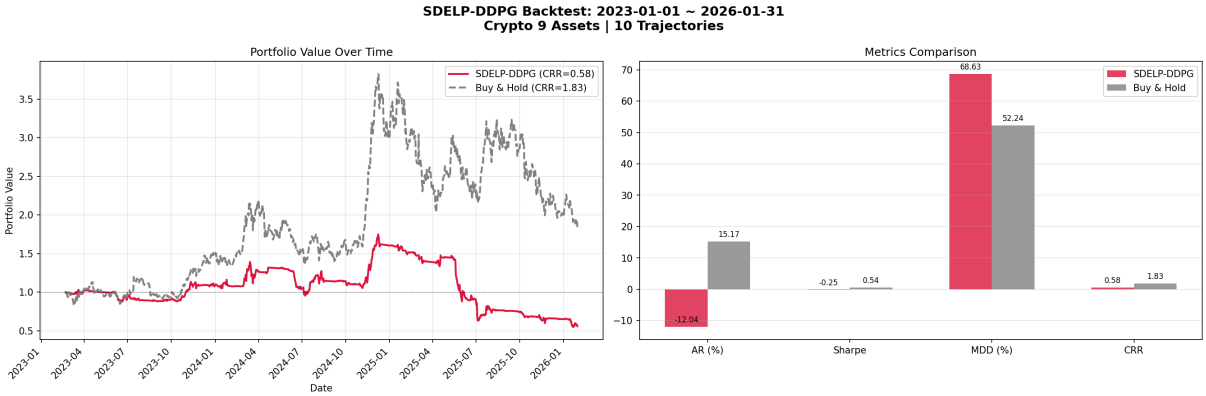

Figure 5: 백테스트 결과 — 좌: 테스트 기간 포트폴리오 가치 곡선 (날짜 X축), 우: 주요 지표 비교 바 차트

10.5 Figure 5: `backtest_results.png` — 백테스트 결과 (별도 실행)

`python backtest.py` 실행 시 생성됩니다.

좌측: Portfolio Value Over Time (날짜 X축)

- **빨간선**: SDELP-DDPG의 테스트 기간 포트폴리오 가치 (CRR = 0.58)
- **회색 점선**: Buy & Hold 벤치마크 (CRR = 1.83)
- **X축**: 실제 날짜 (2023-01 ~ 2026-01)
- **논문 연결**: 논문 Fig. 8에 대응. CRR 값이 범례에 표시됨
- **실제 관찰**: 본 실험에서는 SDELP-DDPG(CRR = 0.58)가 Buy & Hold(CRR = 1.83) 대비 **열위** — 10 에피소드의 짧은 학습으로 인한 과적합. 학습 기간(2018–2022)에서는 2822배 수익을 달성했으나 테스트 기간에 일반화되지 못함
- **논문 기대치**: 충분한 학습(100+ 에피소드) 시 상승장에서 벤치마크 대비 초과 수익, 하락장에서 Cash 비중 증가를 통한 방어력 확보

우측: Metrics Comparison (바 차트)

- AR(%), Sharpe, MDD(%), CRR 4개 지표를 SDELP-DDPG vs Buy & Hold로 비교
- **논문 연결**: 논문 Table 2, Table 3의 수치를 시각화한 것
- **실제 관찰**: SDELP-DDPG — AR = −12.58%, Sharpe = −0.25, MDD = 58.93%, CRR = 0.58. Buy & Hold — AR = 19.17%, Sharpe = 0.56, MDD = 52.24%, CRR = 1.83. 모든 지표에서 Buy & Hold가 우위
- **원인**: 10 에피소드 학습으로 인한 과적합. 학습 데이터(2018–2022)에 과도하게 적응하여 테스트 기간(2023–2026)에 일반화 실패
- **논문 기대치**: 충분한 학습 시 Sharpe Ratio와 CRR에서 벤치마크 대비 우위. MDD에서는 논문에서도 뚜렷한 우위를 보이지 못함 (Section 4.3 한계점)

11 핵심 기여점 및 토론

11.1 수식에서 코드로의 핵심 변환 요약

Table 2: 논문 수식 → 코드 변환 핵심 요약

수식	수학적 표현	코드 구현
식 (1)	$v^t = x^t/x^{t-1}$	`returns = prices[1:] / prices[:-1]`
식 (2)	$w^{t-1} = \frac{a \odot v}{a \cdot v}$	`weights = (w_new * v_full) / dot(v_full, w_new)`
식 (6)	$R = r_t - \beta D_{KL}$	`reward = log_return - beta * kl_penalty`
식 (8)	$a_{k+1} = a_k + u\Delta t + \vartheta(\Delta t)^{1/\alpha} L$	`a_k = a_k + drift + diffusion`
식 (15)	$\nabla_\theta J = \nabla_a Q \cdot \nabla_\theta \Phi$	`actor_loss.backward()` (자동미분)
식 (16)	$a^* = (a + \mathfrak{N})/\sum a_j$	`action = (action + noise) / action.sum()`
식 (17)	$\mathcal{L} = (Q - y)^2$	`critic_loss = MSELoss()(current_q, y)`

11.2 구현상의 핵심 설계 판단

1. **LayerNorm vs BatchNorm:** Drift/Diffusion Net에서 batch=1 추론을 위해 LayerNorm 채택
2. **Diffusion의 입력 고정:** $\vartheta(a_1)$에서 초기 행동 a_1만 사용하여 SDE 발산 방지
3. **Lévy 클리핑:** $[-5, 5]$ 범위로 극단값 제한하여 수치 안정성 확보
4. **Gradient Clipping:** `clip_grad_norm_ = 1.0`으로 그래디언트 폭발 방지
5. **eval/train 모드 전환:** 단일 샘플 추론 시 BatchNorm 통계량 문제 해결

11.3 Session 2의 한계점과 코드에서의 대응

Session 2에서 지적된 한계	코드에서의 대응
Volatility/MDD 우위 없음	보상 함수 식 (6)에 MDD 항이 없음 — 확장 시 보상 재설계 필요
Lévy 과정의 메모리 부족	WINDOW_SIZE=50으로 과거 50일 정보 제공하지만, Lévy 증분 자체는 독립
증분 독립성의 한계	Attention 메커니즘이 SDE 궤적 내에서 시간적 의존성을 부분적으로 보완

— Session 3 끝 —